Análisis de componentes principales (PCA)

Aprendizaje no supervisado · Semana 4

Antonio Miguel López Rodríguez
Sistemas Industriales Inteligentes · MAIIND
Curso 2026–2027

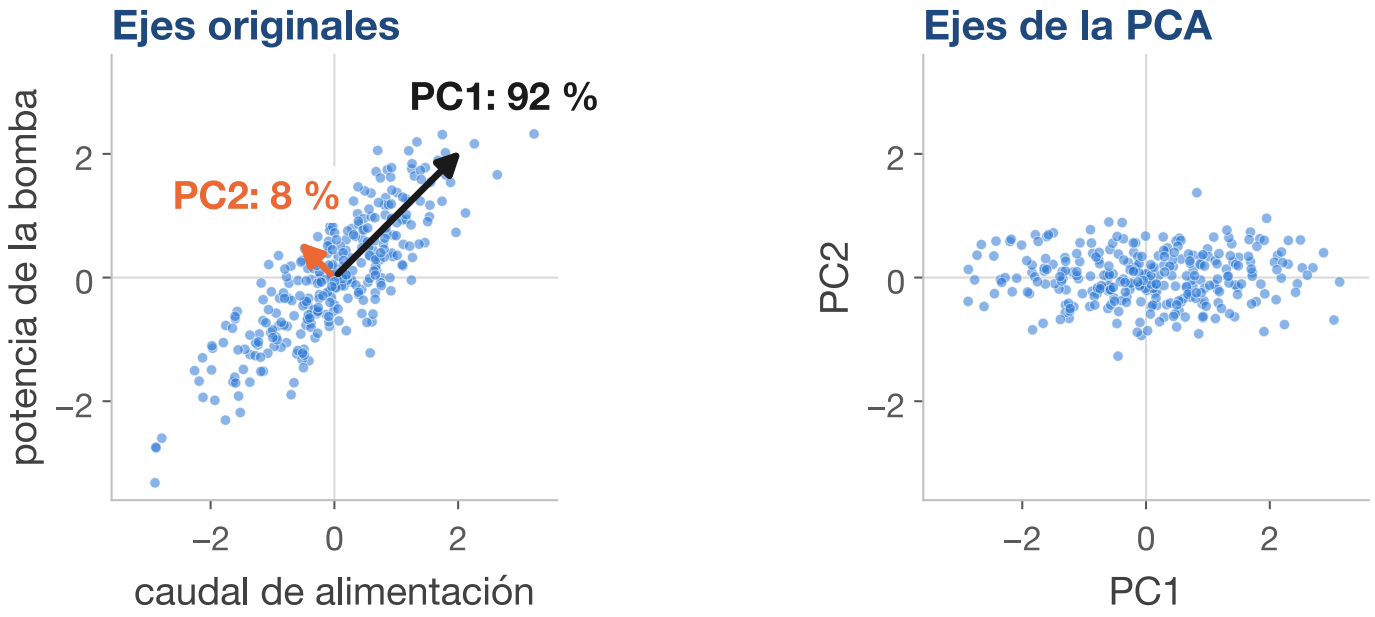
Para qué sirve la PCA

Muchas variables que se mueven juntas contienen menos información de la que parece.

- **Ver** datos de muchas variables en un plano.
- **Resumir** sensores redundantes en unas pocas variables nuevas.
- **Quitar ruido**: quedarse con las direcciones en las que los datos varían de verdad.
- **Preparar** otros métodos: agrupar o entrenar un modelo con menos variables.

Es no supervisada: solo mira las variables de entrada, nunca la salida.

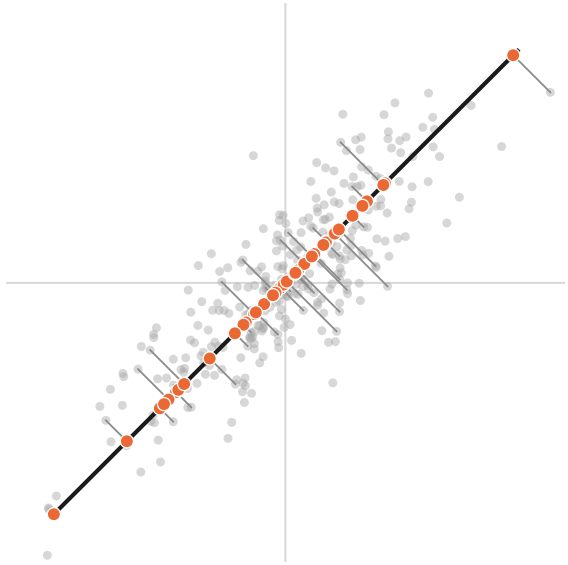
La idea: girar los ejes



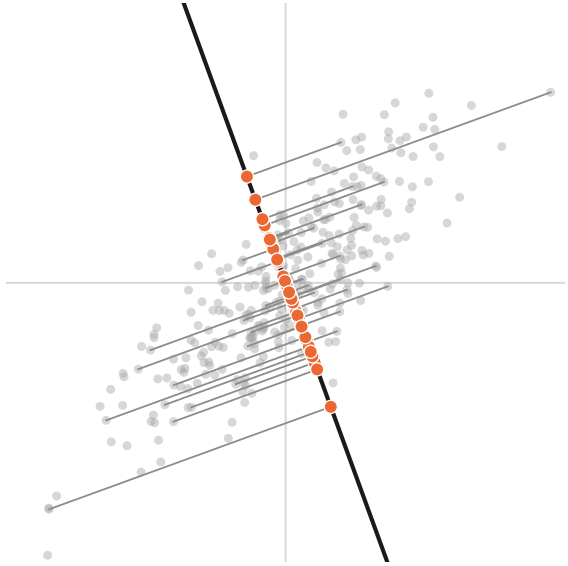
PC1 apunta hacia donde los datos varían más; **PC2**, perpendicular, recoge lo que queda. En los ejes nuevos, las variables ya no están correlacionadas.

¿Qué dirección elegir?

Dirección A: varianza 1,84



Dirección B: varianza 0,47



Proyectar es llevar cada punto a la recta por el camino más corto. La PCA elige la dirección con **más varianza** en las proyecciones, la A, que es también la que pierde menos al proyectar.

Componentes, cargas y puntuaciones

Término	Qué es	En scikit-learn
Componente principal	Una dirección nueva: combinación lineal de las variables	Una fila de `components_`
Carga	El peso de cada variable en una componente	`components_[j, i]`
Puntuación (*score*)	La coordenada de cada fila sobre una componente	`fit_transform(X)`
Varianza explicada	La fracción de la variación total que recoge cada componente	`explained_variance_ratio_`

Hay tantas componentes como variables. Se ordenan de mayor a menor varianza y son perpendiculares entre sí.

Cómo se calcula

1. Estandarizar cada variable: $z = (x - \bar{x})/s$. Las filas forman la matriz Z.
2. Matriz de covarianzas de las variables estandarizadas: $C = \frac{1}{n-1} Z^\top Z$.
3. Vectores y valores propios, $C\,p_j = \lambda_j\,p_j$: cada p_j es una componente (sus cargas) y λ_j, su varianza.
4. Puntuaciones $t_j = Z\,p_j$. Fracción explicada por la componente j: $\lambda_j / \sum_k \lambda_k$.

---

scikit-learn llega al mismo resultado con la descomposición en valores singulares (SVD), numéricamente más estable.

Un ejemplo con 8 sensores

Proceso simulado: dos causas comunes y un sensor independiente

El proceso simulado

Siguen la carga de la planta

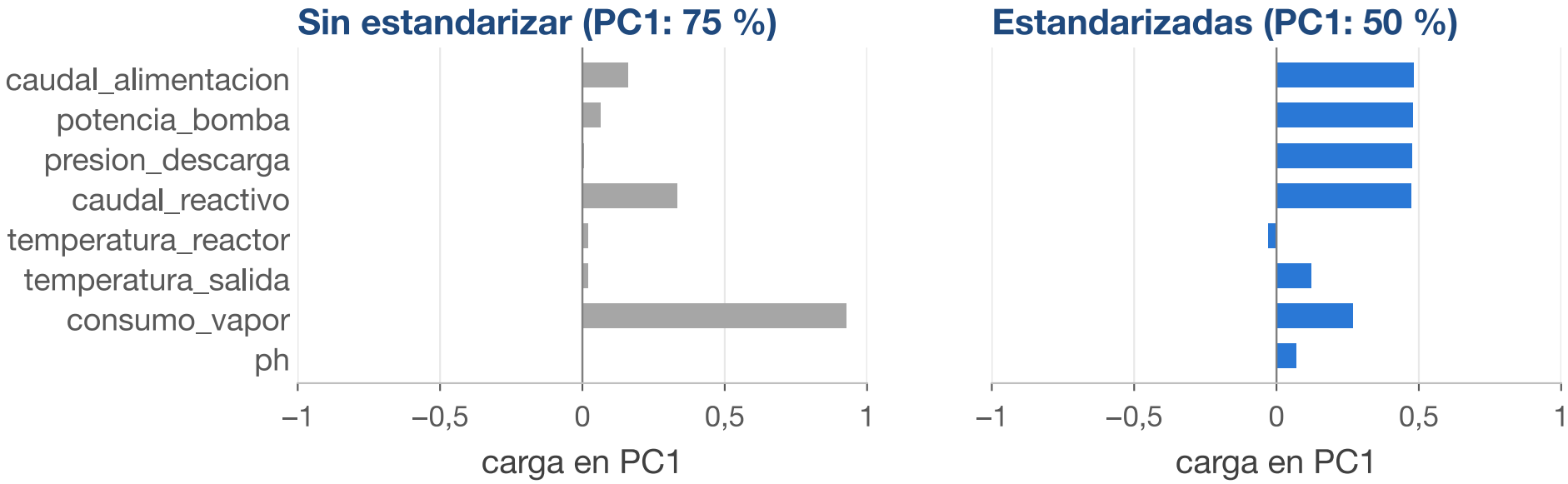
- `caudal_alimentacion` (m³/h)
- `potencia_bomba` (kW)
- `presion_descarga` (bar)
- `caudal_reactivo` (L/h)

Siguen la temperatura

- `temperatura_reactor` (°C)
- `temperatura_salida` (°C)
- `consumo_vapor` (kg/h), que también sube con la carga

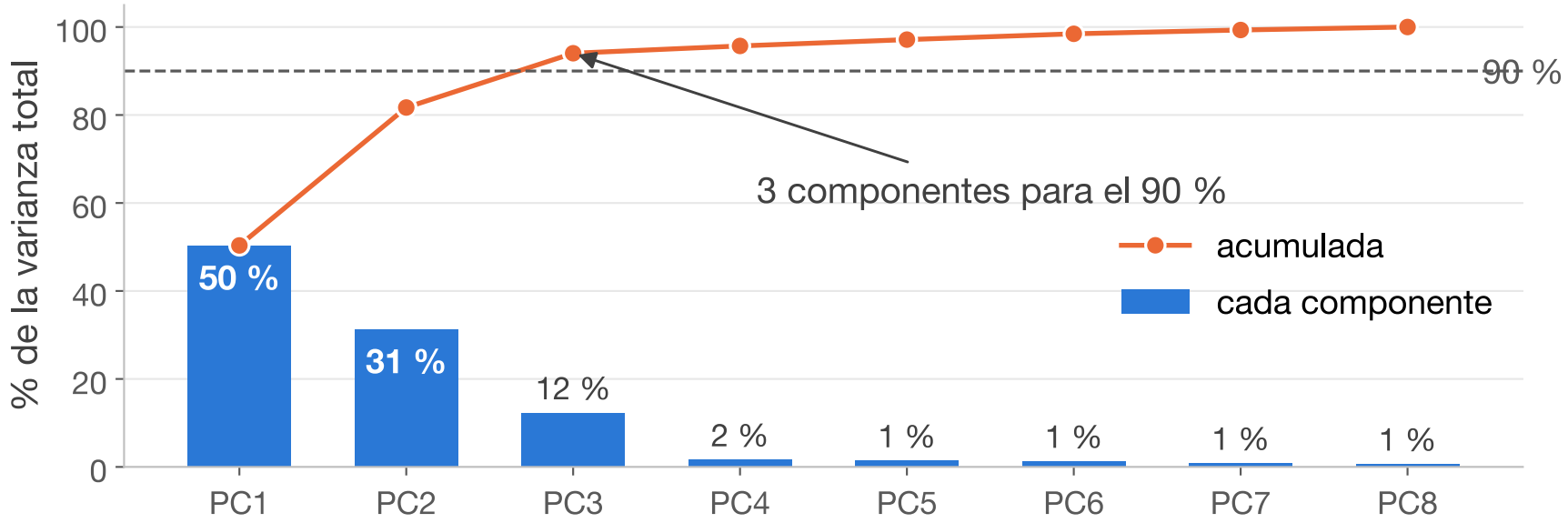
Y un `ph` que no depende de nada. Son 600 muestras de tres regímenes de operación. La PCA no conoce ni las causas ni el régimen: solo ve los 8 sensores.

Primero, estandarizar



Sin estandarizar, PC1 es casi solo `consumo_vapor`, el sensor con números más grandes (desviación típica de 59 kg/h; la del pH es 0,15). La PCA buscaría las unidades, no la estructura.

¿Cuántas componentes?



Criterios habituales: un porcentaje acumulado (del 80 al 95 %), el codo de las barras o el uso (dos, para dibujar). Si las variables apenas están correlacionadas, la PCA comprime poco.

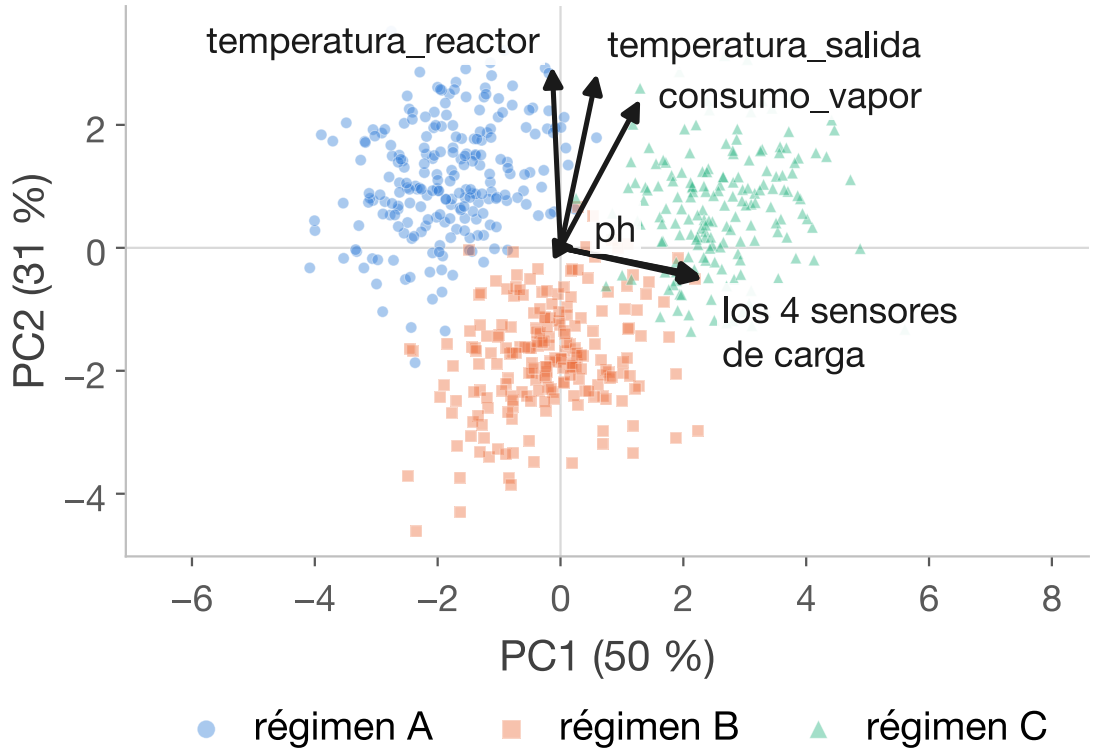
Qué significa cada componente: las cargas

	PC1	PC2	PC3
caudal_alimentacion	0,48	-0,11	-0,02
potencia_bomba	0,48	-0,10	-0,05
presion_descarga	0,47	-0,09	-0,04
caudal_reactivo	0,47	-0,10	-0,02
temperatura_reactor	-0,03	0,61	-0,02
temperatura_salida	0,12	0,58	0,01
consumo_vapor	0,27	0,50	-0,02
ph	0,07	0,00	1,00



- **PC1**: los cuatro sensores de carga, con el mismo peso. Es «la carga de la planta».
- **PC2**: las dos temperaturas y el vapor. Es «la temperatura».
- **PC3**: casi solo el pH, que va por libre.
- El signo de una componente es arbitrario. Importa qué variables pesan y si van en el mismo sentido.

El plano PC1–PC2



- Cada punto es una muestra, dibujada con sus puntuaciones en PC1 y PC2.
- Cada flecha es un sensor: sus cargas en PC1 y PC2. Indica hacia dónde crece.
- Los sensores de carga apuntan hacia PC1; las temperaturas y el vapor, hacia PC2. El pH apenas aparece: está en PC3.
- Colorear por una variable que la PCA no ha usado, aquí el régimen, ayuda a interpretar el plano.

PCA en Python

```
escalador = StandardScaler()
X = escalador.fit_transform(datos[variables])

pca = PCA()
puntuaciones = pca.fit_transform(X)

varianza_explicada = pca.explained_variance_ratio_
varianza_acumulada = varianza_explicada.cumsum()
cargas = pd.DataFrame(pca.components_,
                      columns=variables)
```

`PCA()` calcula todas las componentes.

`PCA(n_components=2)` se queda con dos; `PCA(n_components=0.90)`, con las necesarias para el 90 %.

`cargas`: una fila por componente y una columna por variable.

Importaciones: `StandardScaler` de `sklearn.preprocessing` y `PCA` de `sklearn.decomposition`.

Limitaciones

- **Es lineal**: solo encuentra direcciones rectas; no desenrolla estructuras curvas.
- **Sensible a la escala y a los atípicos**: estandarizar y revisar los datos antes.
- **Las componentes mezclan variables**: para interpretarlas hay que mirar las cargas.

Si después se va a predecir, el escalador y la PCA se ajustan solo con los datos de entrenamiento.